Homework 2: Median Filter

Background

Analyzing a series of images of the (exact) same location at different times is known as "temporal processing". One example of a temporal filter is a *median* filter.

Given several images, a **temporal median filter** takes a list of pixel channel values at each coordinate and calculates the median. This removes outliers and can act to remove unwanted elements (i.e., bad image data) from an image (given that enough *good* image data is provided).

In general, to calculate the median of a list of numbers of odd length, you do the following:

1. Sort the list
2. Calculate the middle location. If we have a list of length n, where n is odd, the location of the middle value is $m = \frac{n+1}{2}$.
3. Return the value at that middle location.

Note

Python lists are indexed at 0, so for a list `num_list` and middle location m, the median value is `num_list[`$m + 1$`]`. Furthermore, Python lists are mutable and can be sorted in place:

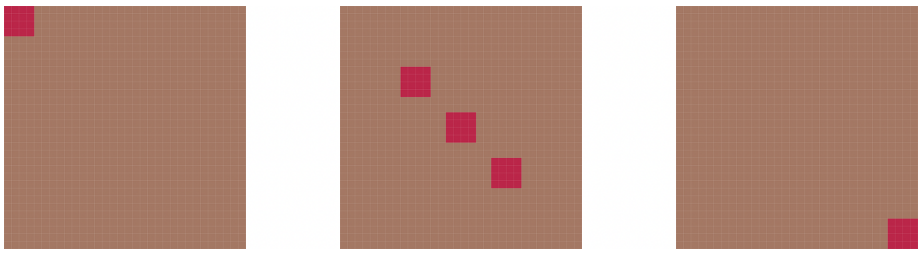
```
my_list = [2, 4, 6, 237, 1]
my_list.sort()
print(my_list)  # prints [1, 2, 4, 6, 237]
```

Task 1 (15 points)

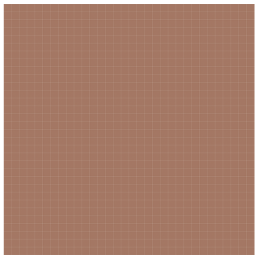
Write a function called `my_median()` that takes a list of integers and returns the median of the list. For this assignment, the list will contain an **odd** number of integers. The `my_median()` function is very important for Task 2 and Task 3. Test it to make sure it works!

Task 2 (25 points)

Use your median function from Task 1 to calculate the median of the red, green, and blue values at each pixel for the images: `box1.png`, `box2.png`, and `box3.png`. The images can be found in the `task2_images` directory in the `sp26_hw2_images directory` (downloaded here). At each coordinate, create a new pixel from the three median calculations. (**Note:** The pixel outlines are added for emphasis and the images are magnified.)



Your result should be:

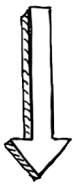


To check if your work is correct, you can verify that every pixel in your result image has RGB value `(164, 120, 100)` — Pantone Color of the Year 2025 *Mocha Mousse*.

Task 3 (60 points)

Suppose you took several pictures of a beautiful location and want to remove an individual walking through the frame. The individual is in all of the images.

Use your median function once again from Task 1 to take the median of the red, green, and blue values at each pixel. With **Pillow**, use the calculated median values of each color channel at each pixel to create a new pixel to place in a new image (of the same width and height as the original images). The basic idea is presented in stylized format below — all of the actual images have the same width and height.



Note: The images for Task 3 can be found in the `task3_images` directory of the `sp26_hw2_images` directory.

Important

Your program will be tested on different images — do not hardcode the number of images or the image dimensions. Your function to calculate the median should be called `my_median()`

Do not use median functionality from packages such as `numpy` or `statistics`. Write your median function such that it only works for sequences of odd length.

Call your output images `task2.png` and `task3.png`, respectively. These images should have the exact same width and height of the source images. If your code bears excessive resemblance to another student's code, you will both receive zero credit and will be reported to the Department Chair.

Deliverable

Submit all source code files and your output images (`task2.png` and `task3.png`). All source code files should include header comments (e.g., name, class, date). Do not submit the source images.

You should be able to use the same code from Task 2 to complete Task 3. The only difference should be the loading of the images. Here is a partial example of what your code might look like:

```
"""
Your header comments
"""
from PIL import Image
from glob import glob

img_list = []

def load_images(location, type):
    for image in glob(f'{location}/*.{type}'):
        img_list.append(Image.open(image))

# Task 1 - Code for my_median() function

# Task 2
# load_images('task2_images', 'png')

# Task 3
# load_images('task3_images', 'png')

# Code to perform median filter and generate new image.
```

Cover image and Task 3 images courtesy of "Isle of Dogs", 2018, Fox Searchlight Pictures